

2.4 Refinements to improve clarity and efficiency

وضاحت بہتر کرنے کے لیے معنی خیز نام، ماڈیول، درست indentation اور مختصر تبصرے استعمال کریں۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 10 of 16

2.4 Refinements to improve clarity and efficiency

LEARN IT

Improving an algorithm means reviewing it step by step: simplifying logic, removing redundant work, and picking smart data structures. A clear algorithm is easier to follow; an efficient one wastes fewer steps.

2.4.1 Refinements for clarity

1. **Use meaningful names.** Replace vague variables with descriptive ones. Bad: `a = b / c`. Good: `average = totalSum / totalItems`.
2. **Break into modules.** Divide a massive block into functions, for example separating `calculateSum()` from the printing logic.
3. **Improve structure and indentation.** Packed loops and IF statements confuse the reader. Spacing reduces mental effort.
4. **Add comments.** Keep them short. Explain **why** something is done, not just what is happening, especially for tricky logic.

2.4.2 Refinements for efficiency

1. **Reduce unnecessary steps.** In Bubble Sort, after the first pass the largest value has settled at the end. Ignore that sorted tail on the next pass by shrinking the inner loop, and you save comparisons.
2. **Use better algorithms.** Searching 1 million **sorted** records with Binary Search instead of Linear Search changes the time from seconds to milliseconds.
3. **Avoid re-computing values** (memoization / caching). If a value never changes, compute it once outside the loop, store it, and reuse it.
4. **Use appropriate data structures.** Direct access → array. Hierarchical data → tree. Do not force one structure everywhere.

WORKED EXAMPLE

Worked example 1 — shrinking Bubble Sort's inner loop ($n = 5$). A naive inner loop always runs $n-1 = 4$ times, for 4 passes: 16 comparisons. The refined inner loop runs 4, then 3, then 2, then 1: 10 comparisons.

```
n = 5
naive = 0
for i in range(n - 1):
    for j in range(n - 1):
        naive = naive + 1

refined = 0
for i in range(n - 1):
    for j in range(n - 1 - i):
        refined = refined + 1

print(naive, refined)
```

OUTPUT
16 10

Worked example 2 — do not re-compute a constant inside the loop.

```
A = 3
B = 4
N = 5

# BAD: C is recalculated every time
for i in range(1, N + 1):
    C = A + B
    print(C * i)
```

OUTPUT
7
14
21
28
35

```
A = 3
B = 4
N = 5
# GOOD: compute C once
C = A + B
for i in range(1, N + 1):
    print(C * i)
```

OUTPUT

7
14
21
28
35

Same output, fewer additions. That is caching: compute once, reuse N times.

REMEMBER IT

Clarity: **N-M-I-C** (Names, Modules, Indentation, Comments). Efficiency: **S-A-C-D** (Shrink steps, Algorithm choice, Cache, Data structure).

COMMON MISTAKE

Mistake: “optimising” by renaming variables and thinking that is an efficiency refinement.

Fix: meaningful names improve **clarity**. Efficiency refinements change the **amount of work** (fewer loop iterations, a better algorithm, no re-compute, a better structure).

اردو میں سمجھیں

وضاحت بہتر کرنے کے لیے معنی خیز نام، ماڈیول، درست indentation اور مختصر تبصرے استعمال کریں۔ کارکردگی کے لیے فضول قدم ہٹائیں (بیل سارٹ میں ترتیب شدہ دم نظر انداز کریں)، بہتر الگورتھم چنیں (لینیئر کی جگہ بائنری سرچ)، لوپ کے اندر وہی حساب بار بار نہ کریں (Caching/Memoization)، اور مناسب ڈیٹا سٹرکچر رکھیں۔

CHECK YOURSELF

1. Rewrite $a = b / c$ with meaningful names for an average.
2. After Bubble Sort's first pass, what should the next pass ignore?
3. In the A+B example, where should $c = A + B$ be written?

Answers: 1. $\text{average} = \text{totalSum} / \text{totalItems}$. 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A clarity refinement is:

- a. Rename, split, comment, simplify logic
- b. Add more nested loops for fun
- c. Shorten names to a,b,c
- d. Remove all spaces

MCQ 2. An efficiency refinement is:

- a. Cut extra work (e.g. already-sorted inner of bubble)
- b. Add random sleeps
- c. Copy the list 12 times
- d. Print inside the inner loop 1e6 times

MCQ 3. Flagged bubble sort's early exit is:

- a. Efficiency refinement
- b. A new UX colour
- c. A foreign key
- d. A pie

MCQ 4. Extracting a function is:

- a. Clarity (and sometimes reuse) refinement
- b. Always worse Big O
- c. OSI layering
- d. A histogram

MCQ 5. Do not "optimise" first if:

- a. The algorithm is still incorrect or unreadable
- b. $n=10^{12}$ already proven
- c. Tests all pass and profiler points here
- d. It is $O(n!)$ on huge n and you must ship

MCQ 6. Redundant work means:

- a. Computing the same thing twice
- b. A needed loop
- c. A primary key
- d. A caption

Extra short questions

1. Two clarity refinements.
2. Two efficiency refinements from the book.
3. A refinement that hurts clarity for 1% speed is:
4. How do you know a refinement worked?

Extra long question

1. Show bubble sort before/after: (1) meaningful names (2) last-i unsorted bound (3) swapped flag. State what each changed (clarity vs efficiency).

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Better names; extract module; remove duplicate logic; add pre/post comments.
2. Stop inner bubble as the suffix grows sorted; avoid extra copies; fewer conditions in the inner loop.
3. Usually a bad deal in XI/XII and in teams.
4. Same tests (correctness) + fewer operations or clearer reading.

Long

1. Code/pseudocode both versions. (1) clarity (2) fewer comparisons $n^2/2$ still $O(n^2)$ (3) best case $O(n)$. Tests unchanged.

Next lecture: 2.5 Concept of Data Structure